

User Relationship Graph Engine for NASSAQ

محرك العلاقات بين المستخدمين داخل منصة نَسَق NASSAQ

ليكون المحرك المسؤول عن إدارة NASSAQ | داخل منصة نَسَق User Relationship Graph Engine أريد إنشاء وفهم وربط العلاقات بين جميع الكيانات البشرية والتعليمية داخل النظام.

هذا المحرك يجب أن يكون من أهم المحركات الأساسية في النظام، لأنه هو الذي يجعل المنصة لا تتعامل مع المستخدمين كحسابات منفصلة فقط، بل ككيانات مترابطة داخل منظومة تعليمية حقيقية.

الهدف من هذا المحرك هو بناء خريطة علاقات ذكية وديناميكية بين:

- الطالب
- ولي الأمر
- المعلم
- مدير المدرسة
- المدرسة
- الصف
- الفصل
- المادة
- الحصة
- السنة الدراسية

بحيث يستطيع النظام فهم:

- من يرتبط بمن
- ما نوع العلاقة
- ما نطاق هذه العلاقة
- ما الصلاحيات الناتجة عنها
- ما البيانات التي يجب أن يراها كل مستخدم بناءً على علاقته بالآخرين

User Relationship Graph Engine الهدف من 1)

الهدف من هذا المحرك هو تحويل النظام من نموذج بسيط يعتمد على

User Account فقط

إلى نموذج أكثر ذكاءً يعتمد على

Identity + Role + Relationship + Context

بمعنى أن النظام لا يسأل فقط

- من هو هذا المستخدم؟

بل يسأل أيضًا:

- ما دوره؟
- بمن يرتبط؟
- ما نوع هذه العلاقة؟
- في أي مدرسة أو فصل أو سنة دراسية تعمل هذه العلاقة؟
- ما الصلاحيات الناتجة عن هذه العلاقة؟

لماذا هذا المحرك ضروري جدًا؟ (2)

بدون هذا المحرك ستصبح العلاقات داخل النظام مبنية بشكل بدائي وصعب التوسع، وسيظهر تعقيد كبير في حالات مثل:

- ولي أمر لديه أكثر من ابن
- طالب لديه أكثر من ولي أمر
- معلم يدرس أكثر من فصل
- طالب ينتقل من فصل إلى فصل
- معلم هو أيضًا ولي أمر لطلاب
- مدير مدرسة يرى فقط البيانات داخل مدرسته
- معلم يرى فقط طلاب الفصول التي يدرس لها
- ولي أمر يرى فقط أبنائه وليس كل الطلاب

لذلك يجب أن يكون هذا المحرك هو المرجع المركزي للعلاقات داخل النظام.

المفهوم العام للمحرك (3)

Graph-Based Relationship Layer هذا المحرك يجب أن يعمل كمحرك

أي أنه يتعامل مع النظام على أساس

Nodes

العُقد أو الكيانات

مثل:

- User Identity
- Student
- Parent
- Teacher
- Principal
- School
- Class
- Section
- Subject
- Session
- Academic Year

- Term

Edges

الروابط أو العلاقات بين الكيانات

مثل:

- parent_of
- child_of
- teaches
- assigned_to
- belongs_to
- enrolled_in
- manages
- studies_in
- attends
- linked_to
- responsible_for

الكيانات الأساسية التي يجب أن يغطيها المحرك (4)

4.1 User Identity

يمثل الهوية الأساسية للمستخدم.

4.2 Parent

يمثل دور ولي الأمر.

4.3 Student

يمثل دور الطالب.

4.4 Teacher

يمثل دور المعلم.

4.5 Principal

يمثل دور مدير المدرسة.

4.6 School

يمثل المدرسة.

4.7 Grade

يمثل الصف الدراسي.

4.8 Class / Section

يمثل الفصل.

4.9 Subject

يمثل المادة.

4.10 Session

يمثل الحصة.

4.11 Academic Year / Term

يمثل السياق الأكاديمي الزمني.

أنواع العلاقات الأساسية (5)

Relationship Types

يجب أن يدعم المحرك أنواعًا متعددة من العلاقات.

5.1 Parent ↔ Student

Parent → Student

نوع العلاقة:

- parent_of
- guardian_of
- responsible_for

Student → Parent

نوع العلاقة:

- child_of
- linked_parent

يجب أن يسمح النظام بأن يكون:

- ولي أمر واحد مرتبط بعدة طلاب

- الطالب الواحد مرتبط بعدة أولياء أمور

5.2 Student ↔ School

Student → School

- enrolled_in_school

School → Student

- has_student

5.3 Student ↔ Grade / Class

Student → Grade

- belongs_to_grade

Student → Class

- belongs_to_class

5.4 Teacher ↔ School

Teacher → School

- employed_at
- assigned_to_school

5.5 Teacher ↔ Subject

Teacher → Subject

- teaches_subject

5.6 Teacher ↔ Class

Teacher → Class

- teaches_class
- homeroom_for
- assigned_to_class

5.7 Principal ↔ School

Principal → School

- manages_school

- principal_of

5.8 Parent ↔ School

علاقة غير مباشرة من خلال الأبناء، لكن يمكن إنشاء علاقة تشغيلية مثل

- linked_to_school_via_children

5.9 Teacher ↔ Student

علاقة غير مباشرة، لكن النظام يحتاج فهمها مثل

- teaches_student
- evaluates_student
- supervises_student

وهذه العلاقة غالبًا تُستنتج من

Teacher → Class

Student → Class

خصائص كل علاقة (6)

Relationship Attributes

كل علاقة يجب ألا تكون مجرد رابط فقط، بل يجب أن تحمل بيانات توضيحية.

مثال:

ParentStudentRelationship

يجب أن يحتوي على:

- relationship_id
- parent_user_id
- student_user_id
- relationship_type
- priority
- is_primary_guardian
- can_receive_notifications
- can_view_financial_data
- can_view_academic_data
- can_pickup_student
- status
- created_at
- updated_at

TeacherClassRelationship

يجب أن يحتوي على:

- relationship_id
- teacher_user_id
- class_id
- subject_id
- academic_year_id
- term_id
- weekly_load
- status

ماذا يفعل هذا المحرك فعليًا؟ 7)

Core Responsibilities

7.1 إنشاء العلاقات

عند إنشاء طالب أو ولي أمر أو معلم أو فصل، يقوم المحرك بإنشاء الروابط المطلوبة تلقائيًا.

7.2 تحديث العلاقات

إذا تغير فصل الطالب أو المدرسة أو ولي الأمر، يقوم المحرك بتحديث العلاقة.

7.3 حذف أو تعطيل العلاقات

إذا تم فك ارتباط ولي الأمر بطالب أو نقل الطالب أو تعطيل معلم، يتم تحديث العلاقة دون كسر النظام.

7.4 الاستعلام عن العلاقات

يجب أن يتيح المحرك استعلامات مثل:

- من هم أولياء أمور هذا الطالب؟
- من هم أبناء هذا ولي الأمر؟
- من هم طلاب هذا المعلم؟
- ما الفصول التي يدرسها هذا المعلم؟
- ما المدرسة التي يديرها هذا المدير؟
- ما الطلاب الذين يحق لولي الأمر هذا رؤيتهم؟

7.5 بناء الصلاحيات من العلاقات

المحرك لا يعطي صلاحيات مباشرة، لكنه يمد محرك الصلاحيات بالمعطيات اللازمة.

مثال:

- active parent_of ولي الأمر يمكنه رؤية الطالب إذا كانت هناك علاقة
- teaches_student المعلم يمكنه رؤية طالب إذا كانت هناك علاقة
- المدير يمكنه رؤية المستخدم إذا كانت علاقته بالمدرسة نفسها قائمة

أمثلة تشغيلية مهمة (8)

Operational Use Cases

الحالة 1 – ولي أمر لديه أكثر من ابن

:ولي الأمر أحمد لديه

- محمد
- يوسف
- سارة

.يجب أن يستطيع النظام إظهار الثلاثة في حسابه، مع بيانات كل واحد منهم بشكل منفصل

الحالة 2 – طالب لديه أب وأم

:الطالب خالد مرتبط بـ

- الأب
- الأم

.يجب أن يستطيع كلاهما رؤية بياناته إذا كانت الصلاحيات مفعلة

الحالة 3 – معلم يرى فقط طلابه

:المعلم يدرّس

- Grade 5A Math
- Grade 6B Science

.يجب أن يرى فقط الطلاب المرتبطين بهذه الفصول والمواد، وليس كل طلاب المدرسة

الحالة 4 – مدير المدرسة يرى ضمن نطاق مدرسته فقط

.إذا كان المدير مرتبطًا بمدرسة الرياض، فلا يجب أن يرى طلاب أو معلمين مدارس أخرى

الحالة 5 – نقل طالب من فصل إلى فصل

:B إلى 5A إذا تم نقل الطالب من 5

- Student → Class يجب تحديث العلاقة
- Teacher → Student غير المباشرة يجب تحديث علاقة
- يجب تحديث الجدول والحضور والتقارير

قواعد السلوك عند تحديث العلاقات (9)

Relationship Update Behavior

عند أي تعديل في العلاقة، يجب أن يقوم النظام بـ

1. التحقق من صحة العلاقة الجديدة
2. حفظ التغيير في قاعدة البيانات
3. تحديث العلاقات المشتقة
4. تحديث الصلاحيات الناتجة
5. تحديث الواجهات والتقارير
6. Audit Log تسجيل العملية في

التكامل مع المحركات الأخرى (10)

Engine Integrations

مع User Relationship Graph Engine يجب أن يتكامل

10.1 User Identity Engine

للحصول على الهوية الأساسية لكل مستخدم

10.2 Role & Permission Engine

لترجمة العلاقات إلى صلاحيات فعلية

10.3 Student Enrollment Engine

لربط الطالب بالسنة الدراسية والصف والفصل

10.4 Teacher Assignment Engine

لربط المعلم بالمواد والفصول

10.5 Communication Engine

لتحديد من يمكن إرسال الرسائل له

10.6 Notification Engine

لتحديد من يستلم الإشعارات

10.7 Attendance Engine

.لتحديد من يملك حق رؤية حضور الطالب

10.8 Reporting Engine

.لتحديد من يرى أي تقرير

10.9 Hakim AI Engine

.لتحليل العلاقات وإنتاج تنبيهات ذكية

11) User Relationship Graph Engine

:يمكن لـ **حكيم** استخدام هذا المحرك لتقديم تحليلات واقتراحات ذكية مثل

- يوجد طالب غير مرتبط بولي أمر
- يوجد ولي أمر لديه 3 أبناء لكن لا يستلم الإشعارات
- يوجد معلم يدرس عددًا كبيرًا من الفصول مقارنة بغيره
- يوجد طالب في فصل غير متوافق مع توزيعه السابق
- يوجد حسابات لها علاقات ناقصة أو غير مكتملة

:أمثلة رسائل حكيم

- "هناك طالبان غير مرتبطين بأي ولي أمر"
- "المعلم أحمد مرتبط بـ 6 فصول، وهو أعلى من المتوسط"
- "لاحظت أن بعض أولياء الأمور لا يملكون صلاحية استلام الإشعارات"

نماذج قاعدة البيانات المقترحة (12)

Suggested Database Structures

Table: user_relationships

.رئيسية للعلاقات Collection يجب إنشاء جدول أو

Fields

- id
- from_entity_type
- from_entity_id
- to_entity_type
- to_entity_id
- relationship_type
- school_id

- academic_year_id
- term_id
- metadata
- status
- created_at
- updated_at
- created_by

أمثلة بيانات

Parent → Student

- from_entity_type = parent
- from_entity_id = P001
- to_entity_type = student
- to_entity_id = S001
- relationship_type = parent_of

Teacher → Class

- from_entity_type = teacher
- from_entity_id = T003
- to_entity_type = class
- to_entity_id = C005
- relationship_type = teaches_class

Student → Class

- from_entity_type = student
- from_entity_id = S001
- to_entity_type = class
- to_entity_id = C005
- relationship_type = belongs_to_class

المطلوبة API واجهات (13)

Required APIs

إنشاء علاقة

POST /relationships/create

تحديث علاقة

PUT /relationships/{id}

حذف / تعطيل علاقة

DELETE /relationships/{id}

أو

PATCH /relationships/{id}/deactivate

جلب علاقات مستخدم

GET /relationships/user/{user_id}

جلب أولياء أمور طالب

GET /relationships/student/{student_id}/parents

جلب أبناء ولي أمر

GET /relationships/parent/{parent_id}/children

جلب طلاب معلم

GET /relationships/teacher/{teacher_id}/students

جلب فصول معلم

GET /relationships/teacher/{teacher_id}/classes

قواعد مهمة جدًا في التصميم (14)

14.1 فقط role لا تعتمد على

الدور وحده لا يكفي لفهم من يرى من

14.2 foreign key لا تعتمد على

العلاقات التعليمية أكثر تعقيدًا من ربط مباشر بسيط

14.3 العلاقات يجب أن تكون قابلة للتغيير زمنيًا

لأن الطالب قد ينتقل من فصل إلى آخر، والمعلم قد تتغير إسناداته

14.4 يجب دعم العلاقات النشطة وغير النشطة

حتى نحافظ على التاريخ

أمثلة على ما يمكننا هذا المحرك من تنفيذه لاحقًا (15)

- ولي أمر يرى أبنائه فقط
- طالب له أكثر من ولي أمر
- مستخدم واحد له أكثر من دور
- مدير يرى ضمن نطاق مدرسته فقط
- معلم يرى فقط طلابه
- إرسال إشعار لولي أمر كل طالب في فصل معين
- عرض الأشقاء تلقائيًا في ملف الطالب
- اقتراحات حكيم المبنية على العلاقات

النتيجة النهائية المطلوبة (16)

هو العقل العلاقتي للنظام، بحيث **User Relationship Graph Engine** أريد أن يكون

- يربط جميع المستخدمين والكيانات
- يفسر العلاقات بينهم
- يمكّن محركات الصلاحيات والتقارير والإشعارات من العمل بدقة
- يسمح للنظام بالتوسع لاحقًا بدون فوضى
- CRUD System يجعل نَسَق منصة تعليمية ذكية حقيقية، وليس مجرد

ويجب أن تكون النتيجة النهائية

- مرنة
- قابلة للتوسع
- ديناميكية
- دقيقة
- مرتبطة بالكامل بقاعدة البيانات
- متكاملة مع جميع محركات النظام الأخرى